```
import numpy as np
def runge_kutta_second_order(f, x0_vector, t0, dt, num_step):

    t = [t0]
    x_state = [np.array(x0_vector)]

    t_1 = t0
    x_1 = np.array(x0_vector)

    for _ in range(num_step):
        k_1 = dt*f(t_1, x_1)
        k_2 = dt*f(t_1 + dt,x_1 + k_1)
        t_2 = t_1 + dt

        x_2 = x_1 + 0.5 * (k_1 + k_2)
        t_2 = t_1 + dt

        t.append(t_2)
        x_state.append(x_2)

        t_1 = t_2
        x_1 = x_2

    return t,x_state

def damped_oscillator(t, state):
    x = state[0]
    v = state[1]
    b = 2.0
    m = 1.0
    k = 1.0
    dxdt = v
    dvdt = (-b/m)*v - (k/m)*x
    return np.array([dxdt,dvdt])

x0_position = 10.0
x0_velocity = 0.0
state_vector = np.array([x0_position, x0_velocity])
t0 = 0.0
total_time = 0.2
num_step = 500

dt = total_time/ num_step
time_values, state_vectors =
runge_kutta_second_order(damped_oscillator, state_vector, t0, dt,
num_step)

position_values = [state[0] for state in state_vectors]
velocity_values = [state[1] for state in state_vectors]
```

```
print(f"\nPosition at t={time_values[-1]:.2f}s (last step):
{position_values[-1]:.4f} cm")
print(f"Velocity at t={time_values[-1]:.2f}s (last step):
{velocity_values[-1]:.4f} cm/s")


Position at t=0.20s (last step): 9.8248 cm
Velocity at t=0.20s (last step): -1.6375 cm/s
```